

Stacks and Queues

Stack — Overview

A stack is a linear data structure that follows LIFO: Last In, First Out. The last element added is the first one removed. Think of a stack of plates — you add and remove from the top only.

Stack Operations

- `push(item)`: add an element to the top — $O(1)$.
- `pop()`: remove and return the top element — $O(1)$.
- `peek()/top()`: view the top element without removing it — $O(1)$.
- `isEmpty()`: check if the stack has no elements — $O(1)$.

Stack Implementation in Python

```
stack = []

# Push
stack.append(10)
stack.append(20)
stack.append(30)

# Pop - removes 30
top = stack.pop()

# Peek
print(stack[-1])  # 20

# isEmpty
print(len(stack) == 0)
```

Stack Use Cases

- Function call management (the call stack) — tracks return addresses and local variables.
- Undo/redo functionality in editors.

- Expression evaluation and syntax parsing (balanced parentheses, postfix/prefix conversion).
- Backtracking algorithms (maze solving, DFS traversal).

Queue — Overview

A queue is a linear data structure that follows FIFO: First In, First Out. The first element added is the first one removed. Think of a line at a ticket counter — first come, first served.

Queue Operations

- `enqueue(item)`: add an element to the rear — $O(1)$ with proper implementation.
- `dequeue()`: remove and return the front element — $O(1)$ with proper implementation.
- `front()/peek()`: view the front element without removing it — $O(1)$.
- `isEmpty()`: check if the queue has no elements — $O(1)$.

Queue Implementation in Python

```
from collections import deque
```

```
queue = deque()
```

```
# Enqueue
```

```
queue.append(10)
```

```
queue.append(20)
```

```
queue.append(30)
```

```
# Dequeue - removes 10
```

```
front = queue.popleft()
```

```
print(queue) # deque([20, 30])
```

Important Note on Python Lists for Queues

Avoid using a plain Python list with `pop(0)` for dequeue — that operation is $O(n)$ because all remaining elements must shift. Always use `collections.deque`, which gives $O(1)$ for both ends.

Queue Variants

- Circular Queue: rear wraps back to the front when space frees up, avoiding wasted space in a fixed-size array.

- Priority Queue: elements are dequeued based on priority, not arrival order (commonly implemented with a heap).
- Deque (Double-Ended Queue): insertion and deletion allowed at both front and rear.

Common Interview Problems

- Valid Parentheses — check balanced brackets using a stack.
- Implement a queue using two stacks (and vice versa).
- Next Greater Element using a stack.
- Sliding window maximum using a deque.
- Min Stack — design a stack that supports `getMin()` in $O(1)$.